

Advanced Data Management

Hackathon: Smart Farm Monitoring & Alerting

1. Scenario

AgriData è un'azienda agricola sperimentale organizzata in tre zone colturali (Zona A, Zona B, Zona C). Ogni zona è dotata di sensori ambientali per il monitoraggio della qualità dell'aria e del suolo, mentre un insieme di componenti di automazione (pompe di irrigazione, ventole e riscaldatori di serra, illuminazione a LED, gateway di controllo) viene monitorato in tempo reale dal punto di vista dei consumi elettrici.

L'azienda vuole superare la gestione manuale dei dati e dotarsi di una piattaforma di data management moderna, capace di raccogliere i flussi di dati generati dai sensori, instradarli attraverso un sistema di ingestion, archivarli nel database più adatto a ciascun tipo di dato, visualizzarli in dashboard interattive, generare automaticamente allarmi in presenza di condizioni critiche per le colture o per gli impianti, e memorizzare statistiche sui dati storici (es. alert e valori orari medi di temperatura e umidità nelle diverse zone).

Nello specifico, il centro di controllo dell'azienda agricola vuole realizzare una piattaforma di data management per soddisfare due diverse esigenze.

1) Ingestion, storage e monitoraggio real-time con alerting

Lo scopo è raccogliere i flussi di dati generati dai sensori, instradarli attraverso un sistema di ingestion, archivarli nel database più adatto a ciascun tipo di dato, e generare automaticamente allarmi in presenza delle condizioni critiche sopra descritte. Le anomalie rilevate possono essere opzionalmente memorizzate in modo persistente e interrogabile (storico degli allarmi, implementazione opzionale). Una dashboard interattiva deve permettere di visualizzare gli stream di dati dei diversi sensori (solo alcuni, a scelta), e lo stato e i consumi degli attuatori (solo alcuni, a scelta).

2) Storage di dati storici per analisi a lungo termine

L'azienda è anche interessata a raccogliere statistiche storiche interrogabili sull'andamento delle zone e degli impianti (es. valori orari medi di temperatura e umidità), in modo che possano essere salvate e analizzate anche a distanza di settimane, mesi o anni, a supporto di analisi predittive e manutenzione preventiva.

Il vostro gruppo è stato incaricato di progettare e realizzare un prototipo funzionante di questa piattaforma, scegliendo autonomamente come combinare le tecnologie disponibili.

2. Obiettivi didattici

- Progettare un'architettura end-to-end di raccolta, trasporto, storage e visualizzazione dati per un caso IoT realistico, con flussi a diversa natura.
- Motivare le scelte architetturali: quale tecnologia usare per quale flusso di dati, e perché.

- Realizzare voi stessi il collegamento tra una sorgente dati 'grezza' (stdout/CSV) e un broker/piattaforma di messaggistica (Mosquitto e/o Kafka), come accade tipicamente quando si integra un sistema esistente in una nuova pipeline.
- Implementare una pipeline funzionante end-to-end, dal dato generato dal simulatore fino alla dashboard.
- Realizzare un sistema di alerting reattivo su condizioni definite, verificandone il comportamento con i dati (anche anomali) generati dal simulatore.
- Lavorare in team su un progetto con vincoli di tempo stretti, tipici di un contesto professionale.

IMPORTANTE: focalizzarsi sulle tecnologie per l'ingestion, la memorizzazione, l'alerting, l'interrogazione e la visualizzazione dei dati. Aspetti quali sicurezza, tecnologie di rete, ecc., sono fuori dallo scopo della prova.

- L'architettura deve permettere di gestire e interrogare i dati nella maniera più semplice, efficace ed efficiente, distinguendo in modo motivato tra flussi ad alta frequenza (serie temporali) ed eventi (allarmi).
- Demandare (ove appropriato) il più possibile le azioni al data management system, riducendo al minimo la parte di programmazione.

Criteri di valutazione

- i. **Progettazione del sistema:**
 - a. Scelta delle migliori tecniche di data management
 - b. Scelte tecnologiche corrette, semplici, efficaci e praticabili
 - c. Corrispondenza alla specifica
- ii. **Completezza e correttezza dell'implementazione**
- iii. **Identificazione dei problemi aperti e di possibili soluzioni**

Il voto è individuale. Ogni membro del gruppo sarà responsabile di una parte del progetto, e dovrà essere in grado di spiegare in dettaglio e senza ambiguità i moduli che ha sviluppato e tutto il codice e le query relative; in caso contrario, la valutazione sarà insufficiente. Ogni gruppo deve lavorare individualmente. Non è ammessa la collaborazione fra i diversi gruppi.

Uso dell'AI:

- È consentito usare strumenti di AI come supporto allo sviluppo del codice, alla scrittura delle query, alla configurazione dei componenti, al debugging, e alla comprensione delle tecnologie.
- Il codice generato dall'AI deve essere sempre verificato, compreso e adattato dal team.
- L'AI non sostituisce la progettazione: le scelte architetturali devono essere motivate e coerenti con il problema.
- Non è accettabile integrare codice non compreso o non testato: il team resta responsabile della qualità e della correttezza della soluzione.
- L'uso dell'AI deve essere critico e ragionato.

3. Tecnologie disponibili

L'architettura è libera: non viene fornita una pipeline di riferimento. Sta al gruppo decidere come comporre le tecnologie viste nel corso, motivando le scelte nella relazione finale. È possibile utilizzare altri sistemi e tecnologie purché coerenti coi contenuti del corso.

Nota: il simulatore di sensori è già containerizzato e pronto all'uso (vedi Sezione 4.4): il file `docker-compose.yml` fornito è il punto di partenza a cui aggiungere gli altri servizi (Mosquitto, Kafka, Telegraf, InfluxDB, MongoDB, Grafana...) scelti dal gruppo.

4. Dataset: il simulatore di sensori

È fornito lo script Python `sensor_simulator.py`, che genera in tempo reale 4 categorie di sensori distribuiti sull'azienda agricola. Il simulatore scrive i dati SOLO in due posti: sullo standard output (una riga JSON per lettura) e in un unico file CSV con schema condiviso tra tutti i tipi di sensore. Non pubblica nulla su MQTT o Kafka: collegare questi dati a Mosquitto e/o Kafka è uno dei compiti del vostro gruppo (Sezione 4.5).

4.1 Categorie di sensori

Tipo ("type")	Campi principali	Granularità
air_quality	temperature_c, humidity_pct, co2_ppm, pm25_ugm3, pm10_ugm3	per zona (A/B/C)
soil_quality	ph, ec_dsm, nitrogen_mgkg, phosphorus_mgkg, potassium_mgkg, soil_temperature_c	per zona (A/B/C)
soil_moisture	soil_moisture_pct	per zona (A/B/C)
power_consumption	actuator_id, actuator_type, status, power_w, voltage_v, current_a	per attuatore (6 componenti)

4.2 Formato di output: stdout e CSV

Ogni lettura generata viene (a) stampata su stdout come oggetto JSON su una singola riga, e (b) appesa come riga in un unico file CSV (default: `data/sensor_data.csv`). Lo schema del CSV è l'unione delle colonne di tutti i tipi di sensore: le colonne non pertinenti a una data riga restano semplicemente vuote. L'header viene scritto una sola volta, alla creazione del file; il file viene aggiornato riga per riga (flush immediato), quindi è consultabile in tempo reale anche mentre il simulatore è in esecuzione.

Esempio di riga su stdout (air_quality):

```
{"sensor_id": "air_zoneA", "type": "air_quality", "zone": "A", "timestamp": "2026-07-02T14:32:58.422+00:00", "temperature_c": 23.31, "humidity_pct": 45.86, "co2_ppm": 447.0, "pm25_ugm3": 7.5, "pm10_ugm3": 21.9}
```

Intestazione del file CSV (colonne unificate):

```
timestamp,sensor_id,type,zone,actuator_id,actuator_type,status,temperature_c,humidity_pct,co2_ppm,pm25_ugm3,pm10_ugm3,ph,ec_dsm,nitrogen_mgkg,phosphorus_mgkg,potassium_mgkg,soil_temperature_c,soil_moisture_pct,power_w,voltage_v,current_a
```

4.3 Esecuzione standalone (senza Docker)

Lo script non ha dipendenze esterne: usa solo la libreria standard di Python 3. Per eseguirlo direttamente sulla vostra macchina:

```
python3 sensor_simulator.py --interval 1
```

Parametri principali (tutti opzionali):

Parametro	Descrizione
--interval	Secondi tra un ciclo di letture e il successivo (default: 5). Parametrico: riducetelo per generare dati più rapidamente durante i test degli alert, aumentatelo per una simulazione più realistica.
--csv-path	Percorso del file CSV su cui appendere le letture (default: data/sensor_data.csv).
--duration	Durata totale della simulazione in secondi (default: infinita, Ctrl+C per fermare).
--seed	Seed per rendere la simulazione riproducibile.

4.4 Esecuzione con Docker (consigliata)

Sono forniti Dockerfile e docker-compose.yml: il simulatore funziona subito, senza installare Python o dipendenze sulla macchina host.

Attenzione alle cartelle sincronizzate col cloud: NON eseguite docker compose da una cartella dentro Google Drive, OneDrive, Dropbox o simili (es. un percorso su un drive virtuale come G:\ per Google Drive Desktop). Questi drive sono filesystem virtuali/di rete e spesso non sono compatibili con i bind mount di Docker Desktop (specialmente col backend WSL2): il container scrive correttamente al suo interno, ma i file non compaiono mai sull'host, senza alcun errore. Copiate il progetto in una cartella locale reale (es. C:\hackathon\...) prima di lanciare docker compose.

```
docker compose up --build
```

1. **Vedere lo stdout in Docker Desktop:** aprite Docker Desktop → scheda "Containers", selezionate il container farm-sensor-simulator e aprite la scheda "Logs": comparirà la stessa sequenza di righe JSON prodotta dallo script (equivalente al comando da terminale `docker compose logs -f sensor-simulator`).
2. **Vedere il file CSV:** il docker-compose.yml monta la cartella locale `./data` dentro il container, sul percorso `/app/data`. Il file `data/sensor_data.csv` compare quindi direttamente nella cartella di progetto sull'host e si apre con un qualsiasi editor, Excel o pandas. In alternativa, dentro Docker Desktop potete usare la scheda "Files" del container per navigare fino a `/app/data/sensor_data.csv`, oppure la scheda "Exec"/"Terminal" per lanciare un comando come `tail -f data/sensor_data.csv`.

Attenzione: usate sempre `'docker compose up'` (non `docker run` direttamente sull'immagine). Solo così viene applicato il bind mount definito in `docker-compose.yml`, che è l'unico modo per far comparire il file CSV nella cartella `./data` sull'host.

4.5 Da fare durante l'hackathon: iniettare i dati su Kafka e/o Mosquitto

Il simulatore fornito produce solo stdout e CSV: realizzare il collegamento tra questi dati e Mosquitto e/o Kafka è compito del vostro gruppo, secondo l'architettura scelta. Implementate le funzioni `publish_to_mqtt()` e/o `publish_to_kafka()`, già predisposte nel file come punto di estensione — vengono chiamate automaticamente per ogni lettura generata, insieme alla scrittura su stdout e CSV. Ricordate di installare le librerie necessarie (`paho-mqtt` e/o `kafka-python`, elencate come commento in `requirements.txt`) nell'ambiente in cui gira il codice che pubblica i messaggi.

5. Requisiti funzionali

- **FR1 — Injection & Ingestion:** realizzare il collegamento tra l'output del simulatore (stdout e/o CSV, Sezione 4.5) e Mosquitto e/o Kafka, e acquisire da lì in tempo reale i 4 flussi di dati.
- **FR2 — Storage:** persistere i dati nel database più adatto a ciascun tipo di dato, motivando la scelta (es. serie temporali vs. documenti/eventi o dati storici aggregati).

- **FR3 — Visualizzazione:** almeno una dashboard Grafana che mostri: andamento temporale di almeno due metriche ambientali (es. umidità del suolo e PM2.5), stato e consumi di alcuni degli attuatori, e una vista riassuntiva per zona.
- **FR4 — Alerting:** un sistema di alert attivo su almeno 3 condizioni (vedi Sezione 6), con evidenza che l'allarme scatta realmente sui dati generati dal simulatore.
- **FR5 — Relazione:** un documento breve (massimo 2 pagine) con schema architetturale e motivazione delle scelte tecnologiche (vedi Sezione 7).

6. Sistema di alerting: le condizioni richieste

Il gruppo deve implementare almeno le 3 condizioni di allarme seguenti. Sono liberi la tecnologia con cui vengono valutate e i dettagli esatti delle soglie, purché motivati.

Alert	Condizione suggerita	Razionale
1. Qualità dell'aria critica	pm25_ugm3 > 50 µg/m ³ per almeno 3 letture consecutive nella stessa zona.	Soglia coerente con gli indici di qualità dell'aria (OMS/EPA): oltre questo livello l'esposizione prolungata è dannosa per lavoratori e, in caso di serre, può indicare un problema di ventilazione.
2. Stress idrico del suolo	soil_moisture_pct < 20% in una zona, per almeno 2 letture consecutive.	Al di sotto di questa soglia le colture entrano in stress idrico; l'allarme deve permettere un intervento di irrigazione tempestivo. Il simulatore include eventi di siccità ("dry_spell") pensati apposta per testare questa condizione.
3. Anomalia nei consumi elettrici	power_w di un attuatore acceso (status = "on") supera 1.5× la sua potenza nominale; <u>implementarlo solo per alcuni sensori di corrente.</u>	Individua sovraccarichi, malfunzionamenti o sprechi energetici degli impianti di automazione. Il simulatore inietta eventi di sovraccarico casuali su un attuatore per validare questa regola.

Facoltativo, per chi finisce in anticipo

Alert di correlazione — es. "irrigazione mancante": soil_moisture_pct sotto soglia critica in una zona E la pompa di quella zona risulta status = "off" da più di N minuti → possibile guasto dell'automazione. Richiede di incrociare due sorgenti di dati (umidità del suolo e stato attuatore).

7. Deliverable finali

Alla fine del lavoro, andranno consegnati elettronicamente su Moodle 2 file:

- Zip con tutti i codici usati nel progetto (script di injection verso Mosquitto/Kafka, file di configurazione, query)
- Pdf con una sintetica documentazione del progetto — schema architetturale e motivazione delle scelte tecnologiche (gli schemi potranno essere disegnati a mano e consegnati su foglio al docente)

Demo live a fine sessione e discussione. Il lavoro verrà discusso il giorno stesso.

8. Appendice: file forniti e comandi rapidi

File	Descrizione
sensor_simulator.py	Script del simulatore (solo libreria standard): stdout + CSV, eseguibile con o senza Docker.
Dockerfile	Immagine minimale (python:3.12-slim) che esegue lo script.
docker-compose.yml	Servizio sensor-simulator pronto all'uso, con volume per il CSV e sezione di esempio da estendere con gli altri servizi.
requirements.txt	Dipendenze opzionali (commentate): servono solo quando aggiungerete la pubblicazione MQTT/Kafka.

Comandi principali

```
docker compose up --build      # avvia il simulatore in Docker (consigliato)
docker compose logs -f sensor-simulator  # segue lo stdout da terminale
python3 sensor_simulator.py --interval 5  # esecuzione standalone, senza Docker
```

Buon lavoro!